

SOLID 检查清单				
SOLID 原则	检查问题	AI 设计是否违反	违反说明	修正方案
S-单一职责	有没有类承担了过多职责？	是	User 类作为领域实体（Entity），不仅包含了数据属性，竟然还包含了 register() 和 login() 这种复杂的业务身份认证逻辑，这是典型的“上帝类（God Class）”倾向。	将认证逻辑抽离。User 仅作为单纯的数据实体类（POJO/Entity）。新增 AuthService 类专门负责 register() 和 login() 等鉴权行为。
O-开闭原则	新增需求类型是否需要修改现有代码？	是	Notification 类中直接写死了 sendNotify() 方法。如果未来系统需要扩充通知渠道（比如新增短信通知、微信推送），就必须打开并修改这个类的源代码。	引入策略模式或抽象接口。将发送逻辑抽象为 MessageSender 接口，新增的具体通知渠道（如 SmsSender）只需实现该接口，无需修改 Notification 实体类。
L-里氏替换	子类是否可以替换父类使用？	是	Admin 直接继承了 User（Admin --> User）。但 User 包含 studentId（学号）、campus（校区）和 creditScore（信用分）。系统管理员根本不需要这些学生专属属性！如果用 Admin 替换 User 运行，调用 getCreditInfo() 会引发严重的逻辑错误。	破除 Admin 对 User 的生硬继承。应该抽象出一个更基础的 BaseAccount（仅包含 id, username, password 等共性），让 User 和 Admin 分别继承 BaseAccount。
I-接口隔离	有没有接口太“胖”，包含了不需要的方法？	是	OrderState 接口定义了统一的 handle(..., OrderEvent event)。这意味着像 CanceledState（已取消状态）这种终态，也被迫去面对和处理 ACCEPT（接单）或 SUBMIT_EVIDENCE 等与它毫无关系的枚举事件，导致内部出现大量冗余的判断或空实现。	不强制所有状态处理所有事件。可以将状态处理逻辑进一步拆分，或者在 P2 设计的事件驱动状态机引擎中统一拦截非法事件，避免让末端状态类背负不属于它的事件处理压力。
D-依赖倒转	高层模块是否直接依赖了低层模块的具体实现？	是	User 直接关联并依赖了具体的 CreditSystem 类（User "1" --> "1" CreditSystem）。如果未来学校要求接入第三方信用系统（如芝麻信用或教务处学分系统），高层模块将面临大改。	面向接口编程。定义一个 ICreditService 接口，让 User 依赖于该接口，而 CreditSystem 作为底层模块去实现这个接口。